



An improved framework based on tracking-by-detection for simultaneous estimation of yield and maturity level in cherry tomatoes

Zhongxian Qi, Wenqiang Zhang^{*}, Ting Yuan^{*}, Jiacheng Rong, Wanjia Hua, Zhiqin Zhang, Xue Deng, Junxiong Zhang, Wei Li

College of Engineering, China Agricultural University, Qinghua Rd. (E) No.17, Haidian District, Beijing 100083, China

ARTICLE INFO

Keywords:

Cherry tomato
Yield estimation
Maturity level estimation
Computer vision
Tracking by detection
Deep learning

ABSTRACT

Automated and accurate counting and ripeness level estimation play key roles in the precision management of cherry tomatoes in greenhouses. This study presents an enhanced framework based on tracking by detection to simultaneously estimate the count and ripeness levels for cherry tomato bunches in greenhouses. Ripeness value is determined by the proportional relationship between the number of ripe and unripe fruits in a tomato bunch. To accurately detect fruit kernels in small sizes and blurred ROIs, a lightweight detector called NanoDet is employed. To reduce ripeness estimation errors caused due to shading, the ripeness value with the highest frequency is selected as the estimate when multiple detections of the same bunch are detected. The ripeness values are categorized into five levels and combined with the counting results. The study's results demonstrate that it achieves 92.79% accuracy in counting and 90.84% accuracy in estimating ripeness levels. The improved algorithm has a processing rate of 15 frames per second. These results demonstrate the potential and value of the enhanced method for counting cherry tomatoes and estimating their ripeness levels.

1. Introduction

Tomato is an important crop whose total world production has reached 187 million tons as of 2020 [1]. In China, more and more tomatoes are grown in smart greenhouses. This mode of cultivation requires intensive labor inputs and refined management [2,3]. In factory production, effectively managing cherry tomatoes depends on rapidly and accurately estimating tomato clusters counts and ripeness. However, there is still a lack of methods to automate these tasks, which contrasts with their labor-intensive and cost-increasing manual counterparts. Hence, there is an urgent need for automated methods that are accurate, reliable and capable of counting cherry tomato bunches and estimating ripeness levels in factory production management [4,5].

In the agricultural production environments, the utilization of visual sensors to acquire images is a mainstream and relatively inexpensive approach [2,6,7]. However, extracting crop and environmental information from these images requires taking into full consideration the specific characteristics of the crop's production environment. Fig. 1 illustrates the effects of lighting on changes in brightness and darkness,

the mutual obstruction between fruit clusters, and the potential misidentification of fruit clusters in adjacent ridges. Furthermore, the main stem visible in the figure can obstruct the fruit clusters, consequently impeding the recognition and detection of the individual fruit grains. When it comes to the detection of fruit clusters, their relatively large pixel area in the image facilitates their recognition and detection, but is also makes them susceptible to confusion with tomato clusters on nearby ridges. However, when it comes to ripeness estimation, individually detecting the ripeness of each fruit grain with precision would be not only time-consuming and labor-intensive, but it could also lead to inaccurate detection due to variations in lighting and obstruction [8,9]. Thus, there are still significant challenges in employing visual methods for counting tomato cluster and estimating ripeness [10].

With the rise of computer vision techniques in recent years, researchers have tried to detect fruit and ripeness by various methods. Traditional methods, including directional gradient histogram, various clustering methods, and Hough transform typically rely on features such as color, shape, and texture for detection [11]. However, in complex agricultural scenes, these methods are often lack accuracy due to light

^{*} Corresponding authors.

E-mail addresses: qzxcu@cau.edu.cn (Z. Qi), zhangwq@cau.edu.cn (W. Zhang), yuanting122@cau.edu.cn (T. Yuan), rongjcsz@gmail.com (J. Rong), s20223071396@cau.edu.cn (W. Hua), s20213071236@cau.edu.cn (Z. Zhang), DengXue@cau.edu.cn (X. Deng), cau2007@cau.edu.cn (J. Zhang), liww@cau.edu.cn (W. Li).

<https://doi.org/10.1016/j.measurement.2024.114117>

Received 7 June 2023; Received in revised form 10 December 2023; Accepted 31 December 2023

Available online 5 January 2024

0263-2241/© 2024 Elsevier Ltd. All rights reserved.



Fig. 1. Real-world recognition and detection scenarios.

variations, background interference and occlusion [10]. In contrast, convolutional neural networks are highly effective in extracting object features, allowing them to mitigate the negative effects of the aforementioned issues to a certain extent [10]. Hence, researchers have widely adopted deep learning for detecting and counting crops. Zhang et al. [12] employed a combination of the SIFT image matching method to generate a panoramic view of a holly tree, followed by the utilization of the YOLOX algorithm to detect and count all the whole fruit on the entire tree, demonstrating robustness in dealing with shadows, occlusions, and incomplete contours. Hu et al. [13] initially employed Faster R-CNN to distinguish individual tomato fruits from overlapping tomato clusters. Subsequently, they proceeded to isolate the fruits from the background by using the H and S components in the HSV color space, thereby enhancing the accuracy of final detection. Besides fruit detection, researchers have employed multi-object tracking algorithms to perform fruit or crop counting, particularly in row-monopoly crop production [14,15]. Multi-object tracking (MOT) algorithms are widely applied in various fields, particularly in production setting, and primarily involve analyzing the appearance and quantity of objects captured in video images without prior knowledge [16]. At present, the majority of MOT algorithms adhere to the concept of object detection followed by tracking, which relying on the accuracy of two key components: target detection accuracy and tracker accuracy [17]. The utilization of advanced object detectors can significantly enhance the capability to capture objects in complex environments. However, current trackers have an issue with objective ID conversion, resulting in counting errors that necessitate enhanced measures for improved fruit counting [18]. In fruit ripeness detection, the color feature on the surface of tomatoes serves as valuable information, enabling fast and nondestructive detection of fruit ripeness [4,19]. Therefore, Begum and Hazarika [20] employed a pre-trained deep learning model to classify individual fruit ripeness. The models was further trained on a tomato dataset with three distinct ripeness levels, resulting in a detection accuracy of 97.37 %. In a laboratory experiment, Wan et al. [21] employed computer vision techniques to extract the contours of a number of tomatoes at three ripeness stages. Furthermore, they obtained color feature values for five additional sampling points within each contour, which were subsequently used as inputs for ripeness prediction in a BPNN network. This approach achieved a prediction accuracy of 99.31 %. However, both methods underwent testing in a controlled laboratory setting, and there are still many challenges in the real conditions. Consequently, researchers endeavored to conduct ripeness detection of tomatoes in real environments. An improved YOLOv4 achieved the estimation of ripe and unripe fruits with an accuracy of 94.44 % and a detection rate of 10.71frames per second in a real environment. Wang et al. [22] further enhanced the performance of detecting ripe and green fruits in real environment by improving Faster R-CNN. This modification led to an average algorithmic accuracy of 96.14 %. Nonetheless, all of the aforementioned methods solely detect

the ripeness of individual fruits within the entire image, without attaining an estimation of an entire bunch of tomatoes.

In this study, it is desired to realize both counting and maturity level estimation of tomato clusters. This goal can be realized by employing a tracking-by-detection algorithm. The framework based on tracker by detection can be extended. In this study, we design an additional ripeness estimation module, which can perform the detection of tomato bunch ripeness without compromising the effectiveness of target detection and tracking. In addition, the tracker by detection method objectively suffers from the defect of ID conversion. Therefore, to improve the accuracy of counting results and ripeness estimation, it is necessary to adopt appropriate strategies to reduce false detections and to fuse counting results with ripeness information. Table 1 lists some studies similar to the research in this paper. Fruit location detection and ripeness assessment are commonly employed in the information sensing component of harvesting robots [23,24]. Conversely, detection of specific plant organs is typically employed for acquiring detailed crop information [25]. In recent years, the application of trackers in combination with target detectors for crop counting, yield estimation and density estimation has gained increasing attention [14,15]. Nonetheless, studies simultaneously addressing both fruit yield estimation (quantity statistics) and ripeness estimation are scarce, with only one study identified that achieved quantity estimation and ripeness estimation of an individual tomato by analyzing ripeness information through HSV color space analysis [26]. The paper did not further integrate the quantity information with the ripeness information, but only obtained the two types of information separately. In addition, the maturity information was only analyzed at the qualitative level and no breakthrough was made at the quantitative level. The difference between our study and these studies is that this paper simultaneously realizes whole bunch ripeness estimation and bunches counting of cherry tomatoes. The combination of YOLOv8 and Deep Sort for bunch number estimation has also not been studied. We also applied a very lightweight detector for detecting the number of unripe and ripe tomato kernels, and designed a very simple function to map the number ratio to a ripeness value between 0 and 1. Additionally, we design a separate information fusion module for fusing bunch count information with ripeness information.

In conclusion, this study aims to propose a method for simultaneously counting cherry tomato bunches and estimating their ripeness levels in greenhouses. The advanced YOLOv8 [27] target detector is used to improve the detection accuracy. Followed this, Deep Sort [28] is utilized for ID identification of each bunch. Meanwhile, the lightweight detection network NanoDet [29] is used to detect ripe and unripe fruits within the bunches identified by YOLOv8. The maturity of the entire tomato cluster is represented by the ratio of mature fruits to immature fruits. Subsequently, the region counting method [18] is used to reduce the counting errors due to ID conversion. The ripeness value with the highest frequency in different frames of the same fruit bunch is used as the detection result to reduce ripeness errors. Furthermore, the final ripeness values obtained through the aforementioned method were categorized into five levels from low to high. The ripeness information is fused and processed along clusters IDs information to achieve the estimation of ripeness levels and counting for tomato bunches. Fig. 3 illustrates the reasoning process of our proposed method. The main contributions of this work can be summarized: (1) We extend a framework based on tracker by detection to enable simultaneous counting and estimating ripeness levels for cherry tomato bunches. (2) We develop a simple module for fast estimating the maturity of bunches without affecting counting. (3) We design an information fusion module to effectively mitigate counting errors caused by ID jumps and errors in maturity estimation caused by occlusion.

The rest of this paper is organized as follows: Section 2 describes the image acquisition and data set creation, the detection method used, the tracking method, the maturity estimation methods, and information fusion module. Section 3 shows various experimental results and

Table 1

Some studies related to our research.

Networks model (Algorithms)	Applications	Dataset volume	Data collections locations	Measurement accuracy	References
Color space fusion + AdaBoost	Harvesting robots	156 ripe tomato samples and 877 background samples from 80 images	Controlled greenhouse	96 %(ripe only)	[23]
Mask R-CNN	Harvesting robots	123 images for tomatoes	Controlled greenhouse	92 %	[24]
Variant of Faster R-CNN	Detection of different organs in tomato plant	30,000 images	Greenhouse	fruits: 82 %, flowers: 85 %, stems: 54 %	[25]
YOLOv4-tiny + CSR-DCF(channel spatial reliability-discriminative correlation filter)	Apples counting and estimation of tomatoes	800 images and 20 living videos	Apple orchard	apples and trunk:99 % Apple counting precision: 91 %	[14]
CenterNet + Deep Sort	Estimation of crop density	947 images and 20living videos	Cotton seedling field	Precision:99 % Counting correlation coefficient R^2 :0.967	[15]
Faster R-CNN + Centroid-based tracking + HSV	tomato counting, yield estimation and maturity estimation of tomatoes	895 images	Controlled greenhouse	Precision:89 %, Qualitative analysis of the ripeness of a single tomato	[26]

discussions. Finally, Section 4 concludes this study.

2. Materials and methods

2.1. Data acquisition

The data were collected within Hongfu Agricultural Tomato Production Park in Daxing District of Beijing, China (116.283601 E, 39.603538 N), during the period from December 2021 to March 2022. The greenhouse follows a standard Dutch greenhouse growing pattern and produces cherry tomatoes called Hongfushi. The cherry tomatoes are cultivated in rows, with a spacing of 1.5 m between each row. Each tomato bunch typically contains around 10–12 fruits. For this study, the RealSense D435i (Intel, Santa Clara, America) was employed as the acquisition device, which was mounted on a raised platform cart to capture image and video data. The D435i integrates an RGB camera capable of capturing 1280×720 pixels and video frames at a rate of 30 FPS per second. The D435i features an integrated IMU that employs an accelerometer and gyroscope to determine the camera's positioning and orientation. With a horizontal viewing angle of 69° and a vertical viewing angle of 42° , the RGB camera captures a wide scene. The RealSense D435i camera connects to the PC through USB 3.0, delivering

high-speed data transfer and power. Additionally, Intel offers a software development kit (SDK) that endows developers with powerful APIs and tools for accessing camera data, performing calibration, and implementing computer vision algorithms. The SDK streamlines the development process and simplifies integration with other algorithms. Therefore, we chose D435i as the device for data acquisition. The camera was positioned approximately 0.6 m horizontally from the tomato bunches and 1 m vertically from the rail height. The camera was rotated by 90 degrees to allow a wider field of view in the height direction, thus obtaining images and videos with a resolution of 720×1280 . To ensure sample diversity, images were taken at three periods of the day (morning, midday and afternoon) and various poses of fruit bunches (see Fig. 2).

In the above way, 2127 original images with tomato bunches were collected as the dataset for as the dataset for detecting tomato bunches. 1382 images with various types of tomato bunches were cropped and selected from the collected images as a dataset for estimating ripeness. Both tomato clusters detection and ripeness estimation data were divided into training set and validation set in the ratio of 8:2. We used LabelImg [30] to label these data by manual annotation. Additionally, several videos were also acquired as data for later count and maturity estimation verification. The captured images and video frames were

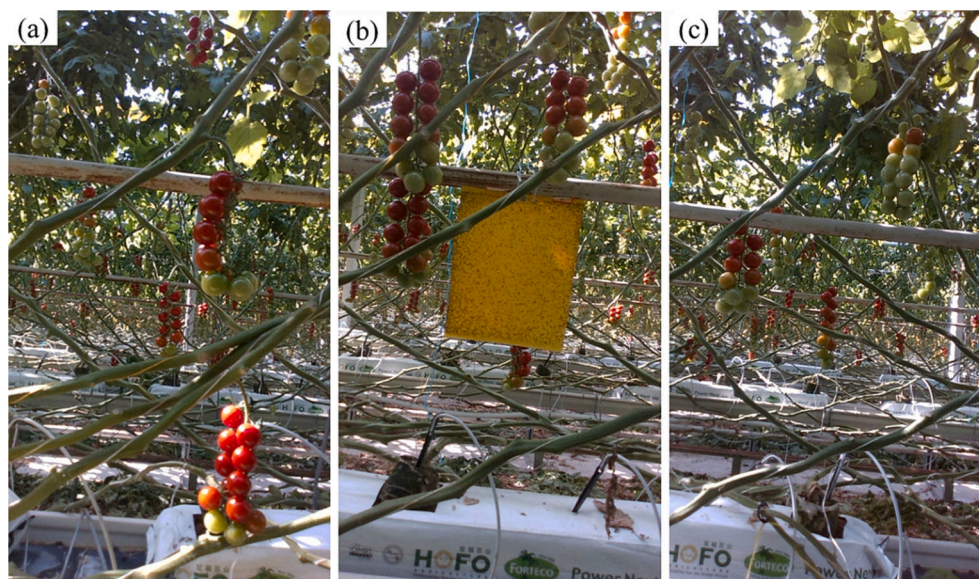


Fig. 2. Images of tomato bunches were acquired in some complex scenarios: (a) illumination influence; (b) fruit overlapping and branch shading; (c) background interference.

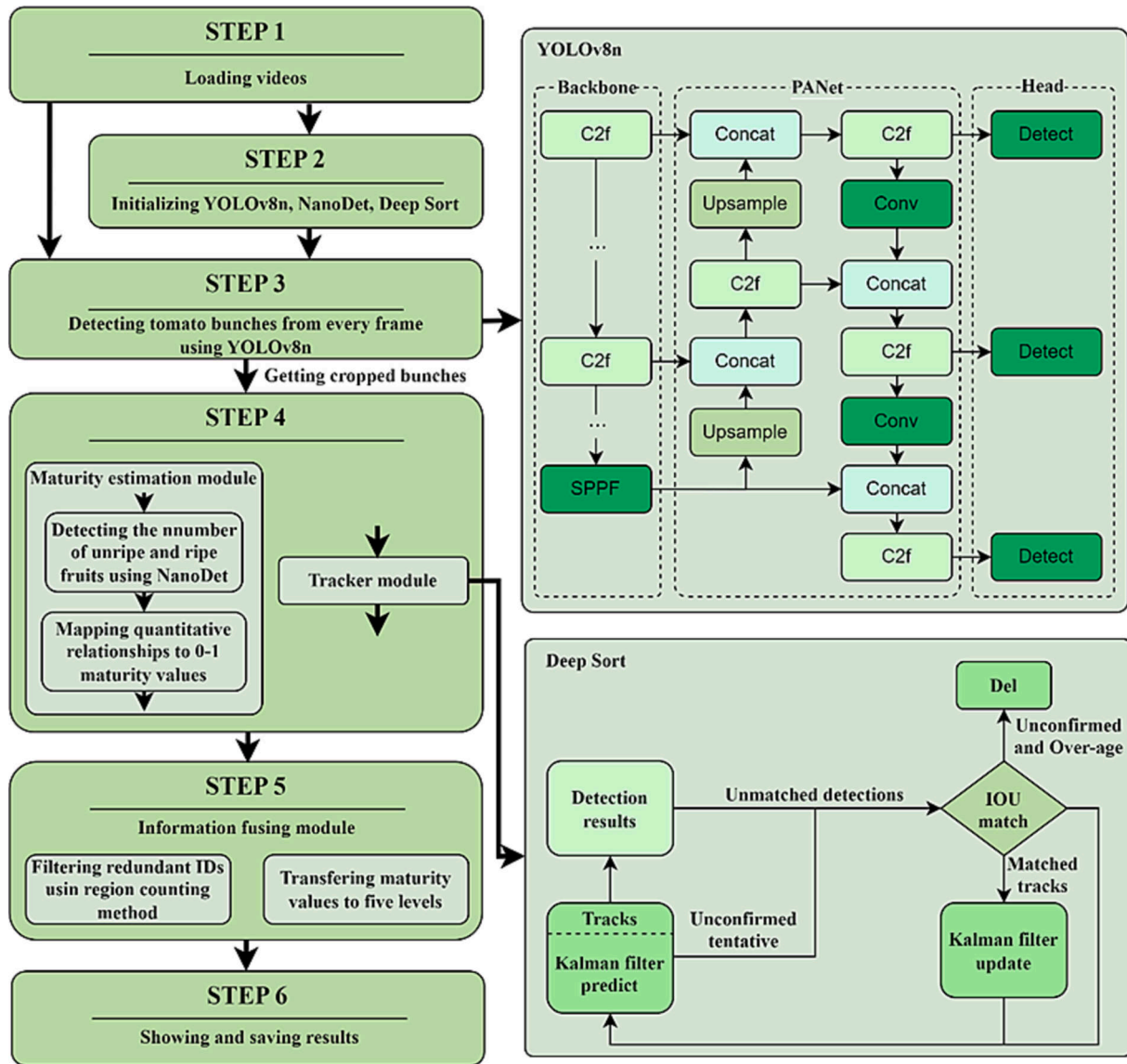


Fig. 3. Overall architecture of maturity estimation and counting, consisting of YOLOv8n detector, object tracker, maturity estimation module, information fusing module.

rotated and processed separately to obtain images and videos with normal view (resolution of 720×1280) for visualization and human eye observation. The resolution of the images used for tomato string detection was resized from 720×1280 to 416×416 . In the ripeness level estimation module, the size of the cropped bunches was uniformly resized to 384×384 or 320×320 . The constructed dataset is summarized in Table 2. The dataset size for fruit bunch detection is 2127, the dataset size for maturity estimation is 1382, and the number of videos for verification is 19.

2.2. The proposed framework of counting and estimating maturity levels for tomato bunches

We proposes a workflow that enables both counting and maturity

grading for tomato clusters. It mainly consisted of the following components: an object detector for detecting tomato bunches, an object tracker, a ripeness estimation module, and an information fusion module. In the object detection module, YOLOv8n is used to mark the fruit bunches in the video frames. The object tracker is used to identify whether the fruit bunches in different video frames are the same fruit bunch and to mark ID numbers. The ripeness estimation module is used to estimate the ripeness value of each bunch. The information fusion module converted the tagged IDs into the number of bunches and fused them with the bunch maturity grades information. The flowchart of the whole framework was shown in Fig. 3. First, sequential frames are inputted into YOLOv8n to generate the detection results. These results are then used as input for NanoDet, which is responsible for identifying ripe and unripe grains. The ratio of ripe to green fruit in the same bunch is calculated to determine the ripeness of the tomatoes within the cluster. Simultaneously, the fruit bunches are processed by the Deep Sort tracker module, which employs techniques such as Kalman filtering, Hungarian algorithm, and ID identification tagging. Next, the tracking results and ripeness values are fed into the information fusion module. A region counting method is used to filter out redundant ID numbers and determine the number of fruit bunches. The ripeness predictions are

Table 2
The summary of constructed dataset.

Images for tomato bunches detection	Images for maturity estimation	Videos for verification
2127	1382	19

classified into five ripeness levels, and the ripeness level that occurs most frequently across different frames for the same fruit cluster is selected as the final ripeness level for the tomato cluster. The redundant IDs filtered out within each frame do not contribute to the ripeness level determination but are removed along with the corresponding fruit bunch IDs.

2.2.1. The object detector for tomato bunches

Accurate bunch detection directly plays a crucial role in determining the effectiveness of subsequent trackers and ripeness estimation of bunches. To enhance counting precision and estimation of ripeness, the current state-of-the-art object detector YOLOv8 is utilized for fruit bunch detection. YOLOv8 builds upon the successful performance of previous YOLO series detectors and inherits YOLOv5 directly. The network structure in the top right corner of Fig. 3 presents the network structure of YOLOv8, in which the C3 module in the original Backbone and Neck is substituted with the C2f module, and the convolution layer in Neck is modified. The prediction head adopts the widely-used decoupled-head and replaces anchor-based with anchor-free method. Additionally, YOLOv8 incorporates various techniques to further enhance its performance, including optimizing the loss function and implementing the task-aligned assigner for sample matching.

2.2.2. Tracking tomato bunches

For the counting task, the accuracy of the counting task is related to how to effectively track the same object appearing in multiple consecutive frames. Due to the potential similarity in appearance among different tomato clusters, it is necessary to assign different IDs to these bunches for trajectory tracking. Nowadays, multi-target tracking strategy based on object detection is the mainstream tracking strategy. The tracker utilizes a Kalman filter to process the trajectories of fruit bunches, and a Hungarian algorithm is employed to estimate their correlation. [31]. However, the Kalman filter alone is not sufficient to cope with changes in the appearance of the same fruit bunch within different frames (see Fig. 6), and significant changes due to occlusion [28]. Deep Sort improves accuracy by calculating the depth features of the bounding boxes of each fruit bunch to enhance the information regarding its action and appearance. This, to some extent, reduces the occurrence of ID jumps when fruit bunches are occluded or experience significant appearance changes across frames. Therefore, we employ Deep Sort to track fruit bunches in different frames. The main processing flow of the Deep Sort algorithm is illustrated in the lower right corner of

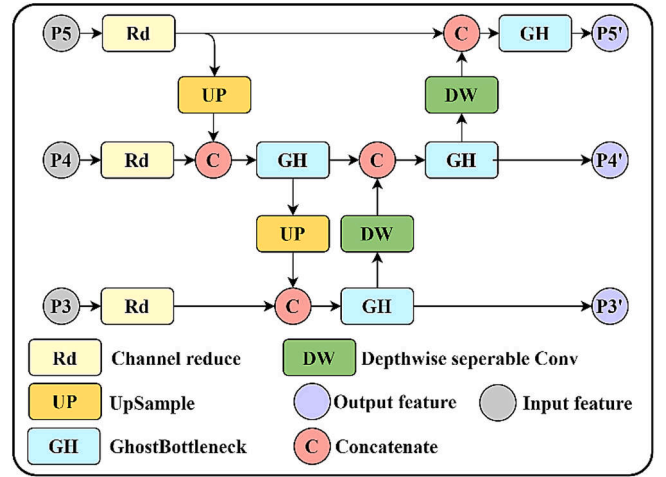


Fig. 5. The structure of GhostPAN. By combining Ghost Bottleneck, channel reduce, depth-separable convolution with PAN, feature information is fully fused while reducing the computation of Neck.

Fig. 3. Initially, the bounding box is detected using YOLOv8n. Then, the tracked objects undergo feature extraction, and the similarity and IOU matching of targets in adjacent frames are computed. If a successful match is found, the information is updated using Kalman filtering, and future position and velocity are predicted. The data is then correlated, and an ID is assigned. If the number of unsuccessful matches exceeds a predefined threshold, the information of the unmatched bounding box is deleted.

2.2.3. Maturity estimation module

Step 4 shown in Fig. 3 illustrates the ripeness estimation module and the tracker module. In the maturity estimation module, the ROIs of the cropped bunches can be obtained after step 3, but not the ripeness of the tomato bunches. Therefore, we designed a module for estimating the ripeness of a tomato bunch. First NanoDet detects the number of ripe and unripe fruits in a fruit bunch, and then obtains the ripeness value of the tomatoes in that bunch using our designed mapping function. We describe the NanoDet detector and the mapping function separately below.

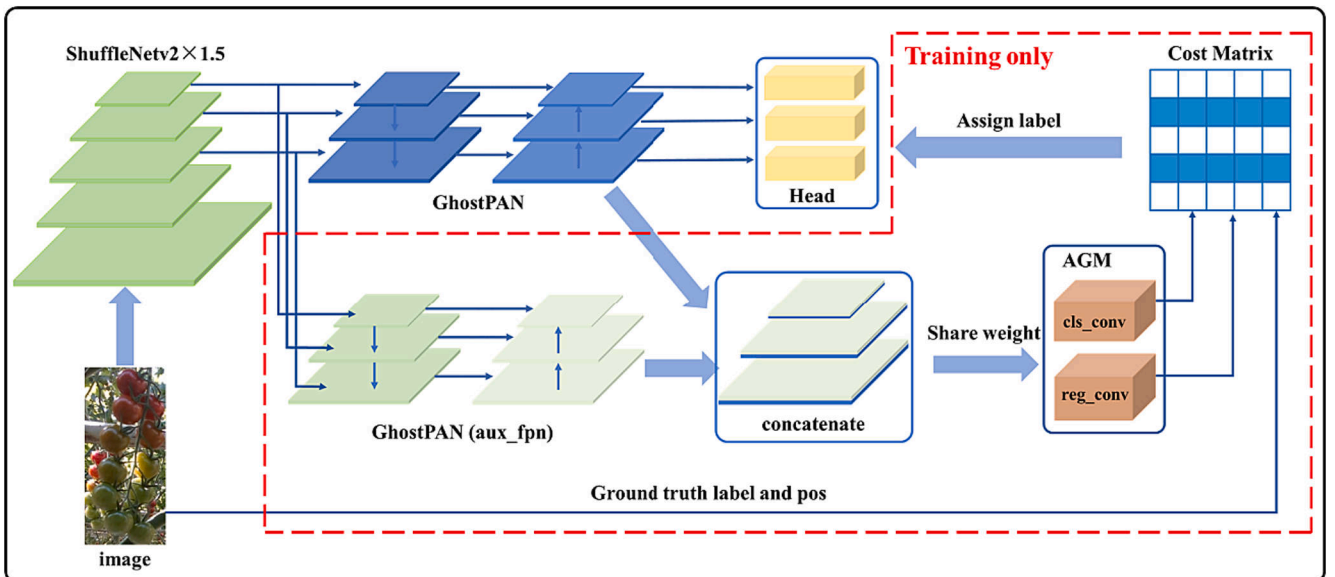


Fig. 4. The structure of NanoDet-m-Plus.

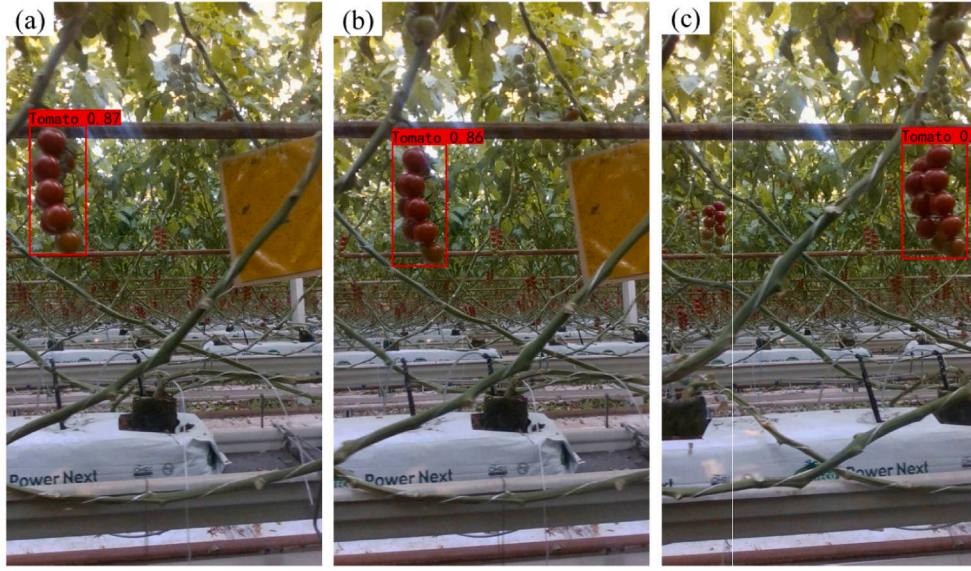


Fig. 6. The same tomato bunch from different frames.

A. Ripe and unripe tomatoes detector

To efficiently estimating the ripeness of the entire tomato bunch, we simplified the process by evaluating the proportion of green and red fruits. Since YOLOv8n detector accurately determines the position of fruits bunches in each frame. Therefore, ripeness detection is conducted in the region of interest (ROI) in each frame, reducing background interference and enhancing the efficiency of detection. The size of the ROI region varies based on the differences in size among the fruit bunches (as illustrated in Fig. 9). This results in potentially blurred outline boundaries of the tomato grains in a cluster, which hinders subsequent detection. Consequently, selecting a detector appropriate for this scenario is necessary. While YOLOv8n can achieve this task, its network architecture is not well-suited for small input images. In this study, we utilize the lightweight target detector NanoDet in conjunction with a ripeness determination method for tomato bunch. Initially, the fruit bunches detected by the tomato bunch detector are fed into NanoDet for the classification of ripe and unripe fruits. Subsequently, the ripeness of the entire tomato bunch is determined based on the ratio of red to green fruits. NanoDet is a lightweight detector consisting of lightweight backbone, FPN, and detection head. The backbone utilizes ShuffleNetv2 [32], a lightweight model. As shown Fig. 5, during the feature fusion stage, Ghost Bottleneck block [33], depth-separable convolution, and reduction of the number of channels of the features are incorporated into PAN [34] to enhance feature fusion and reduce computational requirements.

First, all convolutions in the Path Aggregation Network (PAN) are replaced with 1×1 convolutions to align the feature channel dimensions (Fig. 5 shows this substitution in the Rd module). Bilinear interpolation is used for upsampling to reduce computation, while depth-separable convolution is used for downsampling. In the prediction head, depthwise-separable convolution replaces the normal convolution, and the number of convolution stacks is reduced from four to two. The computation for bounding box regression and classification initially share the same set of convolutions and are subsequently split into two separate processes. During the training stage, the algorithm utilizes a lighter and simpler training assistance module called assign guidance module (AGM) along with dynamic soft label assigner (DSLAs) to address the optimal label matching problem in the lightweight model. In this paper, the chosen version of the model is NanoDet-m-Plus out of several available versions provided by NanoDet. Fig. 4 illustrates the model structure of NanoDet-m-Plus. The red dashed box part in the figure is

solely used for weight updates during training and is not utilized for forward propagation in testing and prediction. This approach significantly enhances the training effectiveness and reduces the inference load.

B. Quantitative relationships mapped to ripeness values

The ripeness composition of a bunch of tomatoes may be as follows, consisting entirely of ripe fruit grains, partially ripe and unripe fruits, and entirely of unripe fruits. In fuzzy logic, a membership function maps input values to a membership value between 0 and 1, indicating the degree of belong to a fuzzy set. In this study, we aim to map the quantitative relationship between unripe and ripe fruits to a ripeness value, which differs from the approach in fuzzy logic. This ripeness value directly represents the ripeness of a bunch of fruits, with the defined scale between 0 and 1. We have developed a function (Eq. (1)) to accurately assess the ripeness of a tomato bunch using the proportional relationship between ripe and unripe fruits. The function maps the detection results to the ripeness value. Indeed, determining the ripeness value of an entire bunch based on the quantitative relationship mentioned above is a simple way. This is possible due to the minimal computational effort required to achieve the desired result. The ripeness estimation module processes the tomato bunches detected in each frame of the previous stage. Sequentially inputting the detected ROIs into NanoDet, the detection results provide labeling information for counting the number of green and ripe fruits in each bunch. The relative proportions between the two categories are mapped to ripening values based on the different cases specified in Eq. (1).

$$M = \begin{cases} \frac{N_r}{N_r + N_g} N_r > 0, N_g > 0 \\ 1.00 N_r > 0, N_g = 0 \\ 0.00 N_r = 0 \end{cases} \quad (1)$$

M denotes maturity, N_r denotes the number of fruits detected as ripe, N_g denotes the number of fruits detected as immature, and the final ripeness is expressed as a decimal between 0 and 1, with closer to 1 indicating higher ripeness and vice versa.

2.2.4. Tracking results and maturity information fusion module

A. Determining the final ripeness value by frequency of occurrence

By the aforementioned methods, we successfully determined the ID information and ripeness of each bunch of tomatoes. Although Deep Sort can reduce the number of ID jumps, overly complex agricultural scenarios will still cause ID jumps [17]. Consequently, this leads to an ID number larger than the actual number of tomato bunches, resulting in a significantly increase in counting errors. However, the utilization of the region counting method [18] can effectively enhance counting accuracy. Furthermore, in the actual detection, there are some instances of both complete and incomplete shading. Additionally, the appearance of the same tomato bunch varies across different viewpoints of frames, as depicted in Fig. 6, resulting in variations in the number of detected bunches. Hence, a reliance solely on the ratio of detected results would introduce a certain degree of detection error. To minimize the error in ripeness detection, we determine the final detection result by selecting the value with the highest frequency of occurrence among the ripeness detection results of the same tomato bunch across different frames.

B. Maturity levels classification

Similar to the quality grading of fruits, it is usually necessary to grade the ripeness because the ripeness grade determines the picking conditions, transportation and storage conditions, and the freshness of tomatoes at the time of consumption [4,35]. Regarding the classification of tomato ripeness levels, there are currently mainstream methods of categorization such as two-level division, three-level division, five-level division, and six-level division [35–37]. These classification methods are only for the ripeness classification of individual fruits. However, when considering the maturity classification for an entire bunch, only whether bunches can be picked or not is taken into account in the case of automatic picking. This is not sufficient for more accurate yield estimation and storage management.

Therefore, in this study, the ripeness level of an entire bunch is classified into one of five ripeness levels based on the distribution of the estimated values between 0 and 1, with reference to the five-level division [37]. In order to provide data support for harvesting and storage management, the ripeness grade of an entire bunch is therefore classified into one of five grades of ripeness. Examples of five grades of tomato bunches are presented in Fig. 7, with two example images listed for each grade. It is clear to see that the proportion of ripe fruit in a tomato bunch gradually increases. This signals the practical application of expressing the ripeness of an entire bunch of tomatoes based on the percentage of ripe fruit. At the time of harvesting, a bunch with a rating of 5 was ready for picking, even though one or two kernels might not be fully ripe (communication with the greenhouse staff also confirmed that a bunch with a rating of 5 was ready for picking). If this maturity scale is combined with the ripening cycle of tomato bunches, it is possible to accurately estimate the number of tomato bunches of each grade in the entire greenhouse. This will provide the exact number of manpower or picking robots needed to pick the bunches that meet the picking requirements in the entire greenhouse, preventing yield loss due to missing the picking period, and providing important data support for more refined

greenhouse management. During picking, both laborers and robots will inevitably damage the fruits. These fruits may not be marketed or stored. Accordingly, non-destructive inspections and yield estimates can be compared to the number of damages, which in turn provides the basis for future yield and transportation and storage management. In addition, since standard commercial cherry tomato production greenhouses can precisely control the crop's nutrient supply, temperature, humidity, light, etc., it is also possible to extrapolate the results of yield estimation to determine whether or not these regulatory measures are justified.

3. Experiments and results

3.1. Evaluation metrics and experiment conditions

The performance of the tomato bunches and grains detection network trained in this study was assessed by mean average precision (mAP). All samples were divided into types according to the combinations of the true and predict class: TP (true positive), FN (false negative), FP (false positive), and TN (true negative). The P (Precision) and R (Recall) of the detection network were defined Eq. (2) and Eq. (3) according to the classification of the samples. AP was an indicator that reflects the global performance of the network, which was calculated in Eq. (4) by P (Precision) and R (Recall). The mean average precision (mAP) can provide a more balanced reflection of the global detection performance of the model. The mAP is the average value of each category (c) of AP, which is defined by Eq. (5).

$$P = \frac{TP}{TP + FP} \quad (2)$$

$$R = \frac{TP}{TP + FN} \quad (3)$$

$$AP = \int_0^1 P(R) dR \quad (4)$$

$$mAP = \frac{1}{c} \sum_{i=1}^c AP_i \quad (5)$$

To evaluate the counting performance of the algorithm, the counting accuracy and the average counting accuracy were respectively defined according to Eq. (6) and Eq. (7).

$$\text{Precision}_{\text{counting}} = 1 - \frac{|\text{Num}_{\text{counting}} - \text{Num}_{\text{True}}|}{\text{Num}_{\text{True}}} \quad (6)$$

$$\text{Acc}_{\text{counting}} = 1 - \frac{\sum_i |\text{Num}_{\text{counting}} - \text{Num}_{\text{True}}|}{\sum_i \text{Num}_{\text{True}}} \quad (7)$$

During training and testing in this article, NVIDIA GPU 2060 6G is used for accelerated computing. The construction model uses Python as the programming language. The hyper-parameters for model training were set as follows. The hyper-parameters of the two object detectors

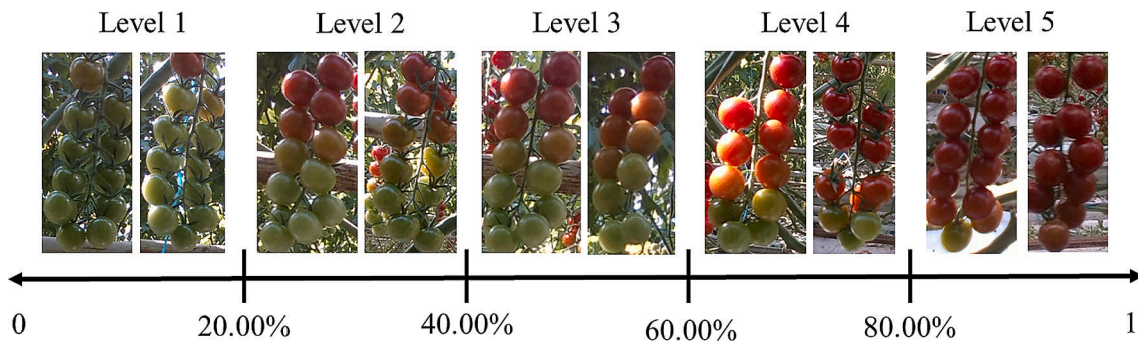


Fig. 7. Examples of different maturity levels.

were the same except for their different input sizes. For the detection model, the hyper-parameters are set as follows: the total epoch was 120, with 60 epoch for frozen training, 60 epoch for unfrozen training, and a batch size of 16. The optimizer used SGD, with an initial learning rate of 0.01, and an attenuation parameter of 0.0005. The momentum and weight decay of the optimizer SGD are set to 0.937 and 0.0005, respectively. In addition, the cosine annealing function is used for the learning rate decline. During training, data augmentation with both mosaic and mixup probabilities of 0.5 is used to improve the performance of the algorithms.

3.2. Detection results

To demonstrate our choice, we select some models for comparison and find that YOLOX_tiny performs best on the test dataset. But instead of choosing it, we chose YOLOv8n because it is only weaker than YOLOX_tiny in various indicators, while reducing the number of parameter and calculation amount by nearly half (as shown in Table 3). Additionally, Table 2 lists the time spent by each model for inferring a thousand images, and it can be seen that YOLOv8n has the shortest time, only 36.93 s. Fig. 8 reveals that the training and validation loss curves of YOLOv8n was converged within 120 epochs. Table 3 informs us that YOLOX_tiny is the best performing algorithm, but YOLOv8n has an accuracy that is only 0.68 % lower than the former, while having the fastest inference speed.

3.3. Tomato bunches counting results

To demonstrate the effectiveness of the tracker and the rationality of the counting method, we conducted simple processing on the video stream acquired in Section 2.1. This resulted in 19 video segments, each approximately 30 s long, with a frame rate of 30fps. The problem of ID offset is present in many tracking algorithms [17]. Therefore, we utilizes the area counting method for the final predictions. In this experiment, we counted the number of real fruit bunches in each video and the number of tomato bunches counted by the program. Table 4 displays the algorithmic count results, manual counting results, and their deviations. By employing Eq. (6) and (7), the average detection accuracy was determined to be 93.43 %. These findings align with Rong's study [18], suggesting that adopting an effective counting strategy can enhance accuracy.

3.4. Bunches maturity detection results

To obtain a better detection accuracy, a total of 1382 different fruit bunches were quantified and their size distribution was plotted on a scatter plot shown in Fig. 9. The horizontal axis represents the height distribution of the bunches in the images, while the vertical axis represents the width distribution of the bunches in the images. The pixel width of the fruit bunches in the images mostly range between 100 and 150, while the height mostly range between 200 and 400 pixels. Therefore, selecting a suitable model has a significant impact on the recognition effect based on the size distribution. This study trained several popular models at the size of 384×384 , as well as the NanoDet model at the original size of 320×320 . The effects of each model is

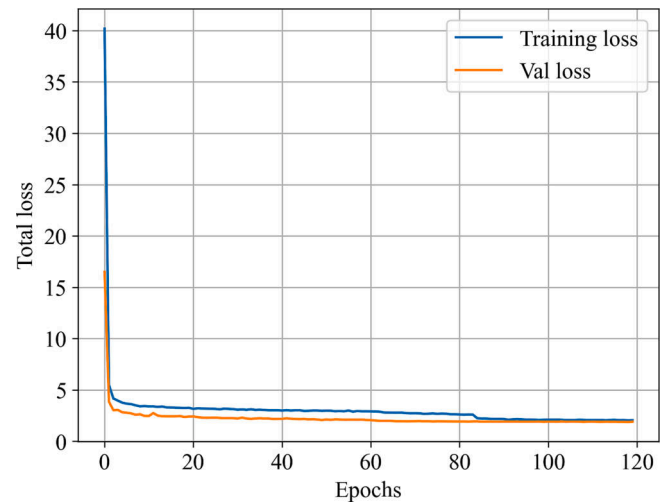


Fig. 8. Training and validation loss curves of YOLOv8n network.

compared, and their respective test results are presented in Table 5. Although the YOLO series models perform well, they exhibit weaker performance than NanoDet at smaller scales. This indicates that NanoDet is more effective in recognition at small scales, and its computational power requirement is also lower.

As shown in Figs. 10 and 11, the tomato grains were detected for both images using different models and the results were visualized. The actual number of grains to be detected in Bunch 1 is 10. Although NanoDet did not detect all the grains, it achieved the best result, detecting 8 grains. In contrast, YOLOv8n and YOLOX_tiny detected 6 and 7 kernels, respectively. In Bunch 2, 10 kernels actually needed to be detected. NanoDet detected all of them correctly, while YOLOv8n detected 9 kernels and YOLOX_tiny detected 11 kernels, misdetecting the stem in the background as fruit grains. For the recognition of fruit grains within the cropped fruit bunch region, the imaging may be blurry, and a model with good performance may not always achieve the expected results for the recognition of small-sized images; instead, NanoDet is more adapted to this detection task. In some cases, the acquired video stream may have degraded imaging due to jittering problems, resulting in local areas where the pixels of object boundaries are not very clear (see Figs. 10 and 11). This adds difficulty to the model detection and recognition. In this case, how to choose a suitable model or improve networks is the main issue.

3.5. Comparison of the results of different maturity update strategies

The selection of an appropriate ripeness update strategy is crucial for obtaining more accurate ripeness results, two strategies for update ripeness values were compared. Specifically, the maturity of the same fruit bunch in multiple frames is determined as the final maturity based on the average maturity value and the maturity value with the highest frequency of occurrence. The number of different ripeness levels was counted on 19 videos and the results of these two methods were compared (Supplementary files Video 10, Video 19). As shown in Fig. 12, in the bar chart, green represents the maturity level quantity

Table 3

Comparison of the performance of different tomato bunches detectors.

Tomato detector	P	R	mAP _{0.5}	mAP _{0.75}	Inference	GFLOPS	Params
CenterNet	97.46 %	92.00 %	97.20 %	85.40 %	74.85 s	46.354G	32.665 M
YOLOv5n	94.27 %	78.93 %	92.40 %	72.60 %	44.15 s	1.785G	1.765 M
YOLOv5s	91.37 %	81.87 %	94.50 %	78.30 %	45.88 s	6.741G	7.025 M
YOLOX_tiny	92.71 %	98.40 %	98.50 %	92.80 %	51.85 s	6.436G	5.033 M
YOLOX_nano	98.51 %	70.40 %	91.60 %	65.30 %	50.93 s	1.077G	896.949 K
YOLOv8n	92.13 %	96.80 %	98.20 %	92.60 %	39.63 s	3.462G	3.011 M

Table 4

The accuracy of counting results.

Clip number	Ground truth	Counting number	Bias	Clip number	Ground truth	Counting number	Bias
1	30	30	0	11	37	33	4
2	31	31	0	12	44	39	5
3	37	33	4	13	45	43	2
4	18	17	1	14	35	35	0
5	22	21	1	15	33	31	2
6	10	10	0	16	24	20	4
7	24	23	1	17	23	21	2
8	24	21	3	18	26	24	2
9	38	33	5	19	20	20	0
10	34	30	4	Total	555	515	40

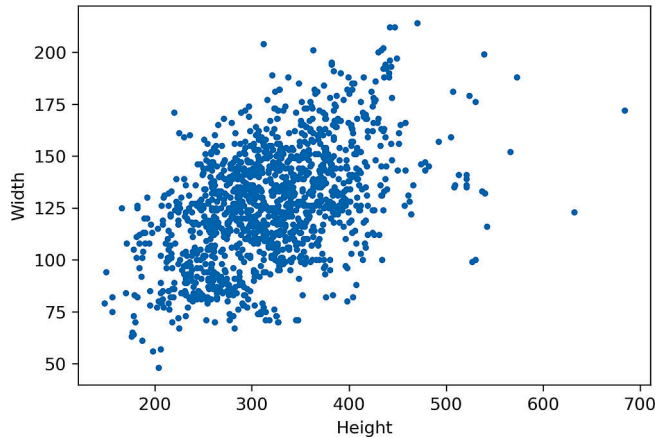


Fig. 9. The distribution of bunches size.

Table 5

Comparison of the performance of different detectors.

Fruits detector	Input size	mAP _{0.5}	mAP _{0.75}	GFLOPS	Params
YOLOv5s	384 × 384	95.10 %	81.40 %	5.744G	7.025 M
YOLOv5n	384 × 384	93.50 %	78.40 %	1.522G	1.767 M
YOLOX_tiny	384 × 384	97.50 %	91.00 %	5.484G	5.033 M
YOLOX_nano	384 × 384	96.90 %	88.30 %	0.917G	896,949 K
YOLOv8n	384 × 384	97.10 %	89.60 %	2.950G	3.011 M
NanoDet-m-Plus	320 × 320	98.00 %	92.30 %	1.75G	2.44 M

statistics after mean processing, blue represents the value with the highest frequency as the statistical result of detection results, and yellow represents the real result of manual statistics. It is obvious that the

statistical result of the blue bar is closer to that of the yellow bar. The final results by highest frequency of maturity values as maturity level obtained 90.84 % accuracy on 19 videos, while the final results by mean values as maturity level obtained only 69.96 % accuracy. If the errors in the counting results were included, the accuracy of the two methods mentioned above was 84.29 % and 64.92 %, respectively. During the statistical process, several factors affecting ripeness grade estimation are found. Different poses of fruit bunches result occlusion between fruit grains, and the detection performance can be effectively improved by the ripeness update strategy. During the cluster ID identification process, bunches with similar postures and maturity levels often results in ID confusion. This means that the model mistakenly classified different bunches as the same, potentially leading to errors in counting and ripeness estimation. While rare, several instances of this issue have been found. The primary contributing factor to this error is shading from bunches and main stems. In this case, the detection of the number of bunches is affected, which in turn affects the ripeness estimation.

In most cases, obtaining ripeness values for the same tomato bunch by different field-of-view angles and using the most frequent value as the final result provides reliable results. This is because it is easier to detect all the grains of the same bunch when inspected on the oblique side, while it is more likely that only some of the grains are detected when inspected on the front side. This, of course, may lead to errors in the results. However, the detection results of different frames are averaged and the result is the average of all detection results by Mean method. Averaging the results tended to incorporate results with larger errors, which may result in larger errors. As shown in Fig. 13, different detection results were presented in different frames for the same fruit bunch with the same ID identification number through different ripeness value updating methods (Supplementary file video 19). For ID 16 bunch, the detection results were 72.91 % and 90.00 % at frame 976 (see Fig. 13a, Fig. 13b), and 41.05 % and 90.00 % at frame 1016 (see Fig. 13c, Fig. 13d), respectively. In comparison, updating maturity values by

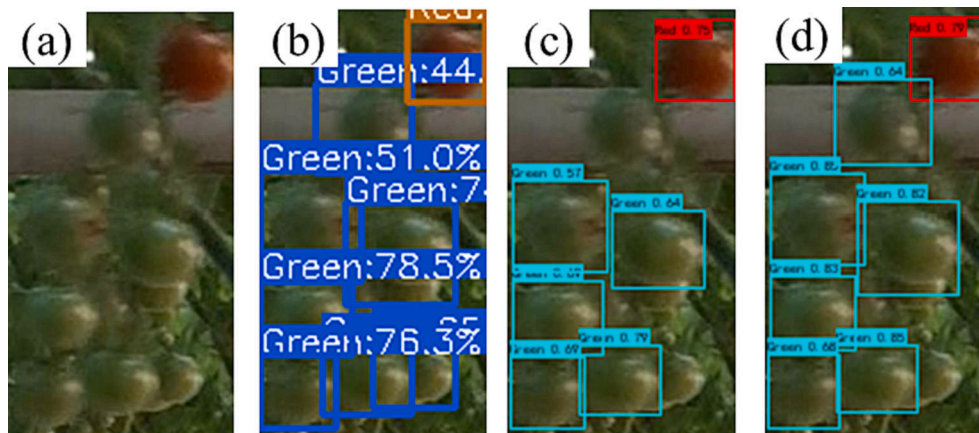


Fig. 10. Visualization of the effect on the fruits detection using different models in Bunch 1: (a) The original image; (b) The output from NanoDet; (c) The output from YOLOv8n; (d) The output from YOLOX_tiny.

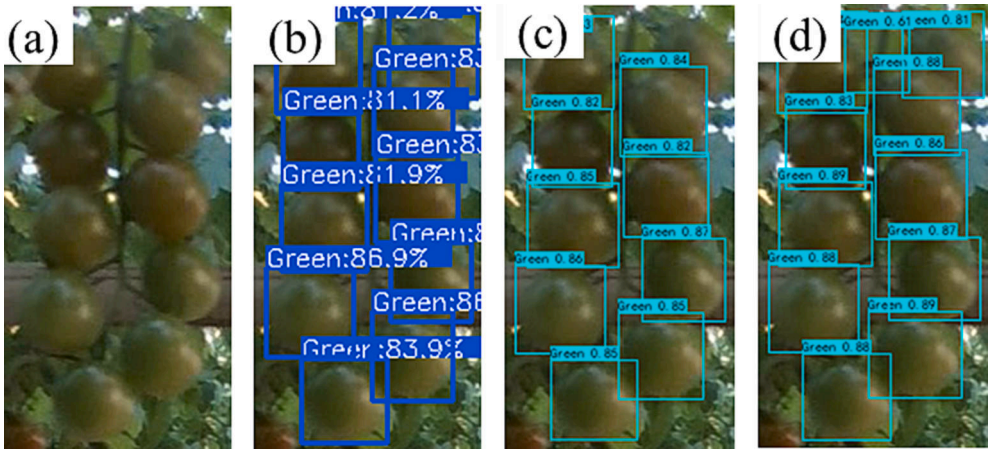


Fig. 11. Visualization of the effect on the fruits detection using different models in Bunch 2: (a) The original image; (b) The output from NanoDet; (c) The output from YOLOv8n; (d) The output from YOLOX_tiny.

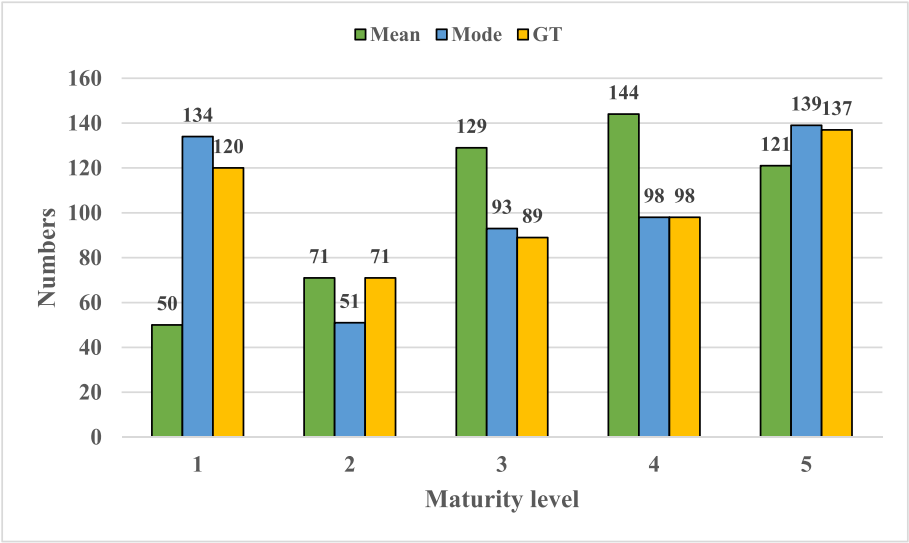


Fig. 12. Comparison of results for different maturity levels based on 19 videos.

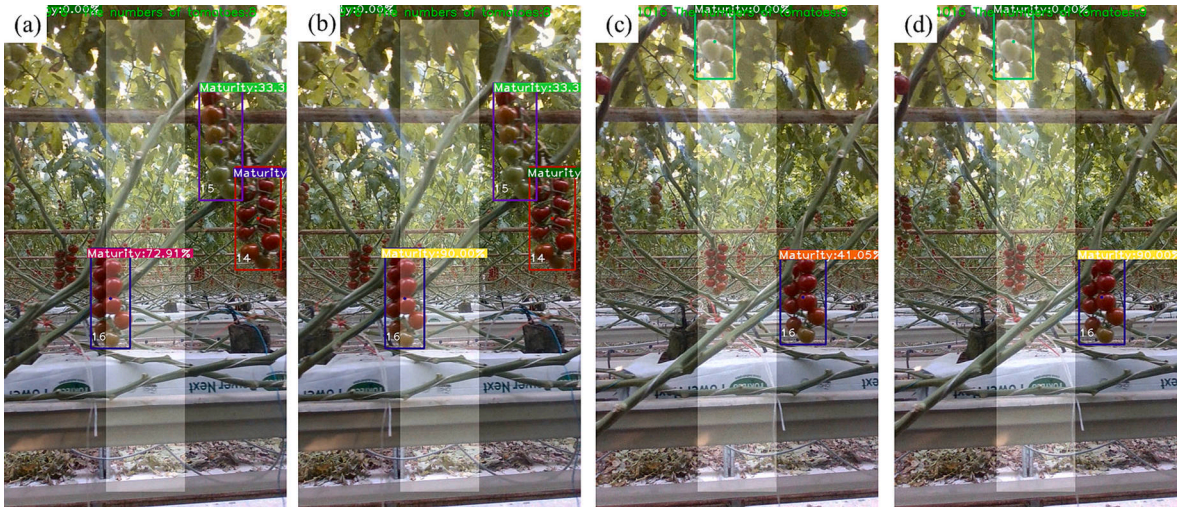


Fig. 13. Examples of ripeness detection results for the same fruit bunch in different frames by different ripeness update strategies in Video V19. (a) the ripeness value of ID 16 at Frame 976 by Mean method; (b) the ripeness value of ID 16 at Frame 976 by Mode method; (c) the ripeness value of ID 16 at Frame 1016 by Mean method; (d) the ripeness value of ID 16 at Frame 1016 by Mode method.

Mode method has better performance.

The final test results done on the NVIDIA RTX 2060 GPU are shown in Table 6. The video frames can run at an average rate of 15 FPS, with an estimated maturity level accuracy of 90.84 % excluding counting errors and 92.79 % counting accuracy.

3.6. Discussion

The current work suggests that expanding the framework based on tracker by detection in estimating the maturity levels and counting for tomato clusters is an effective improvement strategy. During clusters detection, since the proportion of fruit clusters in the whole image is relatively high, occlusion does not cause significant disturbance to the detection of fruit clusters' positions. Therefore, we choose YOLOv8n as the tomato bunches detector, which can have the better balance in terms of operating efficiency and detection performance. In the future, it is possible to attempt improving YOLOX-nano to achieve higher performance while maintaining the lowest computational requirements. This would allow the algorithm to run more smoothly in edge systems.

In maturity estimation, we utilize a lightweight NanoDet to detect the quantity of mature and immature fruits, and then map the relationship of quantity to a maturity value between 0 and 1 using a simple function. The occlusion between the main stem, fruit bunches, and within the fruit bunches is an unavoidable issue when detecting the number of fruit grains. The occlusion between the main stem and fruit bunches may render the fruit grains undetectable, consequently affecting the estimation of maturity. The same fruit bunch may exhibit different postures across different frames (perspectives), which can also result in occlusion within the fruit cluster. Such variations lead to changes in the detected quantity. The occlusion between the main stem and fruit bunches hampers an accurate counting of fruit grain quantity, and altering perspectives may not fundamentally solve this problem. Additional manual intervention may be required, which could increase the cost of crop inspection and management. On the other hand, adjusting the perspective can potentially help detect the accurate quantity of fruit grains by addressing occlusion within a fruit bunch. In the information fusion module, it is necessary to effectively integrate tracking information and maturity values. While Deep Sort can reduce ID switches, it still cannot completely eliminate them. Therefore, this study adopts a region counting method to convert the number of ID numbers into the number of tomato bunches to improve the counting accuracy. At the same time, a maturity value is transformed into one of five levels, and the maturity level is determined based on the frequency of occurrence, which mitigates the maturity estimation error due to the posture variations between the same cluster in different frames and occlusions within the bunch. ID jumps also introduce estimation errors in maturity because of an ID number corresponds to a maturity value. In future work, it is necessary to optimize the estimation results of maturity while considering the issue of ID jumps, which may require further optimization of the tracker.

Tomato clusters cultivated in the facility greenhouse examined in this study exhibit a more standardized growth pattern compared to other agricultural scenarios, with fixed growing heights and distances from the medium. This characteristic has piqued the interest of researchers in the field of automated inspection and harvesting of tomatoes. However, the effectiveness and scalability of deep learning-based vision methods rely heavily on diverse and comprehensive datasets. In agricultural settings, data collection is a time-consuming and labor-intensive process, further complicated by stringent data protection, which makes it challenging to obtain publicly available, high-quality data. Regrettably, the limited dataset we managed to acquire did not fully encompass the complexities associated with tomato growth. Furthermore, during our data collection and experimentation, we encountered the unavoidable issue of vibration in the acquired dynamic video data, resulting in lower imaging quality compared to static images. Therefore, it is imperative to conduct further sensitivity analysis of the proposed method, specifically

Table 6

The final results of the experiment using the NVIDIA RTX 2060 GPU.

FPS	Maturity level estimation accuracy	Counting accuracy
15	90.84 %	92.79 %

investigating the impact of different scene characteristics (such as occlusion and video imaging quality) on bunches counting and ripeness estimation.

4. Conclusion

This study proposes a framework that simultaneously achieves the counting of cherry tomato bunches and the estimation of their ripeness levels. It accurately estimates the yield and distribution of cherry tomatoes in an accurate manner. In the proposed method effectively utilizes the extension of a framework based on tracking by detection. The framework included a ripeness estimation module that categorizes the detected ripeness values into five levels. To mitigate count maturity estimation errors, a fusion module is designed to integrate count information and maturity level information. The proposed method can perform inference at an average speed of 15FPS, with the bunch counting accuracy of 92.79 %, and the ripeness level estimation accuracy of 90.84 % without counting errors (84.29 % with counting errors). The experiments demonstrates the effectiveness of the proposed approach in estimating the number of tomato bunches counting and maturity levels. However, there are some limitations in this study, as occlusion can still interfere with accurate maturity estimation. In addition, there is room for improvement in the overall algorithmic process. In the future, further optimization of the entire algorithm framework and its components can be helpful for accurate yield estimation and maturity level assessment of cherry tomatoes, thereby generating positive impacts on more precise production and storage management.

CRedit authorship contribution statement

Zhongxian Qi: Data curation, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Wenqiang Zhang:** Conceptualization, Funding acquisition, Resources, Supervision, Writing – review & editing. **Ting Yuan:** Resources, Supervision, Writing – review & editing. **Jiacheng Rong:** Conceptualization, Validation. **Wanjia Hua:** Investigation, Validation. **Zhiqin Zhang:** Investigation, Validation, Data curation. **Xue Deng:** Investigation, Validation. **Jun-xiong Zhang:** Conceptualization, Writing – review & editing. **Wei Li:** Conceptualization, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work received funding support from The Fundamental Research Funds for the Central Universities (2020TC017), National Key Research and Development Program of China (2016YFD0701501) and The 2115 Talent Development Program of China Agricultural University (107100109024).

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.measurement.2024.114117>.

References

- [1] H. Sun, M. Ren, J. Zhang, Genome-wide identification and expression analysis of fibrillin (FBN) gene family in tomato (*Solanum lycopersicum* L.), *PeerJ* 10 (2022) e13414.
- [2] S.D. Alwis, Z. Hou, Y. Zhang, M.H. Na, B. Ofoghi, A. Sajjanhar, A survey on smart farming data, applications and techniques, *Comput. Ind.* 138 (2022) 103624, <https://doi.org/10.1016/j.compind.2022.103624>.
- [3] S. Ragaveena, A. Shirley Edward, U. Surendran, Smart controlled environment agriculture methods: a holistic review, *Rev Environ Sci, Biotechnol.* 20 (2021) 887–913, <https://doi.org/10.1007/s11157-021-09591-z>.
- [4] M. Rizzo, M. Marcuzzo, A. Zangari, A. Gasparetto, A. Albarelli, Fruit Ripeness Classification: a Survey, (2022). <https://doi.org/10.48550/arXiv.2212.14441>.
- [5] M.H. Saleem, J. Potgieter, K.M. Arif, Automation in agriculture by machine and deep learning techniques: a review of recent developments, *Precision Agric.* 22 (2021) 2053–2091, <https://doi.org/10.1007/s11119-021-09806-x>.
- [6] O. Acar, H. Sağlam, Z. Şaka, Measuring curvature of trajectory traced by coupler of an optimal four-link spherical mechanism, *Measurement* 176 (2021) 109189, <https://doi.org/10.1016/j.measurement.2021.109189>.
- [7] P. Suebsombut, A. Sekhari, P. Sureepong, P. Ueasangkomsate, A. Bouras, in: The Using of Bibliometric Analysis to Classify Trends and Future Directions on “smart Farm”, Media and Technology (ICDAMT), IEEE, Chiang Mai, Thailand, 2017, pp. 136–141, <https://doi.org/10.1109/ICDAMT.2017.7904950>.
- [8] M. Liu, W. Jia, Z. Wang, Y. Niu, X. Yang, C. Ruan, An accurate detection and segmentation model of obscured green fruits, *Comput. Electron. Agric.* 197 (2022) 106984, <https://doi.org/10.1016/j.compag.2022.106984>.
- [9] Y. Tang, M. Chen, C. Wang, L. Luo, J. Li, G. Lian, X. Zou, Recognition and localization methods for vision-based fruit picking robots: a review, accessed February 24, 2023, *Front. Plant Sci.* 11 (2020), <https://www.frontiersin.org/articles/10.3389/fpls.2020.00510>.
- [10] A. Koirala, K.B. Walsh, Z. Wang, C. McCarthy, Deep learning – Method overview and review of use for fruit detection and yield estimation, *Comput. Electron. Agric.* 162 (2019) 219–234, <https://doi.org/10.1016/j.compag.2019.04.017>.
- [11] A. Gongal, S. Amatya, M. Karkee, Q. Zhang, K. Lewis, Sensors and systems for fruit detection and localization: A review, *Comput. Electron. Agric.* 116 (2015) 8–19, <https://doi.org/10.1016/j.compag.2015.05.021>.
- [12] Y. Zhang, W. Zhang, J. Yu, L. He, J. Chen, Y. He, Complete and accurate holly fruits counting using YOLOX object detection, *Comput. Electron. Agric.* 198 (2022) 107062, <https://doi.org/10.1016/j.compag.2022.107062>.
- [13] C. Hu, X. Liu, Z. Pan, P. Li, Automatic Detection of Single Ripe Tomato on Plant Combining Faster R-CNN and Intuitionistic Fuzzy Set, *IEEE Access* 7 (2019) 154683–154696, <https://doi.org/10.1109/ACCESS.2019.2949343>.
- [14] F. Gao, W. Fang, X. Sun, Z. Wu, G. Zhao, G. Li, R. Li, L. Fu, Q. Zhang, A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard, *Comput. Electron. Agric.* 197 (2022) 107000, <https://doi.org/10.1016/j.compag.2022.107000>.
- [15] H. Yang, F. Chang, Y. Huang, M. Xu, Y. Zhao, L. Ma, H. Su, Multi-object tracking using Deep SORT and modified CenterNet in cotton seedling counting, *Comput. Electron. Agric.* 202 (2022) 107339, <https://doi.org/10.1016/j.compag.2022.107339>.
- [16] Y. Dai, Z. Hu, S. Zhang, L. Liu, A survey of detection-based video multi-object tracking, *Displays* 75 (2022) 102317, <https://doi.org/10.1016/j.displa.2022.102317>.
- [17] J. Villacrés, M. Viscaino, J. Delpiano, S. Vougioukas, F. Auat Cheein, Apple orchard production estimation using deep learning strategies: A comparison of tracking-by-detection algorithms, *Computers and Electronics in Agriculture*. 204 (2023) 107513, <https://doi.org/10.1016/j.compag.2022.107513>.
- [18] J. Rong, H. Zhou, F. Zhang, T. Yuan, P. Wang, Tomato cluster detection and counting using improved YOLOv5 based on RGB-D fusion, *Comput. Electron. Agric.* 207 (2023) 107741, <https://doi.org/10.1016/j.compag.2023.107741>.
- [19] C. Li, Q. Cao, F. Guo, A method for color classification of fruits based on machine vision, *WSEAS TRANSACTIONS ON SYSTEMS*. 8 (2009) 312–321.
- [20] N. Begum, M.K. Hazarika, Maturity detection of tomatoes using transfer learning, *Measurement: Food*. 7 (2022) 100038, <https://doi.org/10.1016/j.meaf.2022.100038>.
- [21] P. Wan, A. Toudeshki, H. Tan, R. Ehsani, A methodology for fresh tomato maturity detection using computer vision, *Comput. Electron. Agric.* 146 (2018) 43–50, <https://doi.org/10.1016/j.compag.2018.01.011>.
- [22] Z. Wang, Y. Ling, X. Wang, D. Meng, L. Nie, G. An, X. Wang, An improved Faster R-CNN model for multi-object tomato maturity detection in complex scenarios, *Eco. Inform.* 72 (2022) 101886, <https://doi.org/10.1016/j.ecoinf.2022.101886>.
- [23] Y. Zhao, L. Gong, B. Zhou, Y. Huang, C. Liu, Detecting tomatoes in greenhouse scenes by combining AdaBoost classifier and colour analysis, *Biosyst. Eng.* 148 (2016) 127–137, <https://doi.org/10.1016/j.biosystemseng.2016.05.001>.
- [24] M. Afonso, H. Fonteijs, F.S. Fiorentin, D. Lensink, M. Mooij, N. Faber, G. Polder, R. Wehrens, Tomato Fruit Detection and Counting in Greenhouses Using Deep Learning, *Front. Plant Sci.* 11 (2020), <https://doi.org/10.3389/fpls.2020.571299>.
- [25] Z. Yuncheng, X. Tongyu, Z. Wei, D. Hanbing, Classification and recognition approaches of tomato main organs based on DCNN, *Dbkxb.* 33 (2017) 219–226, <https://doi.org/10.11975/j.issn.1002-6819.2017.15.028>.
- [26] D. Seo, B.-H. Cho, K.-C. Kim, Development of Monitoring Robot System for Tomato Fruits in Hydroponic Greenhouses, *Agronomy* 11 (2021) 2211, <https://doi.org/10.3390/agronomy11112211>.
- [27] Ultralytics, YOLOv8, (2023). <https://github.com/ultralytics/ultralytics> (accessed April 18, 2023).
- [28] N. Wojke, A. Bewley, D. Paulus, Simple Online and Realtime Tracking with a Deep Association Metric, (2017). <http://arxiv.org/abs/1703.07402> (accessed September 30, 2022).
- [29] Rangliyu, NanoDet-Plus, (2023). <https://github.com/Rangliyu/nanodet> (accessed April 18, 2023).
- [30] Tzatalin, LabelImg-GitHub Repository, (2019). <https://github.com/heartexlabs/labelimg> (accessed May 5, 2023).
- [31] A. Bewley, Z. Ge, L. Ott, F. Ramos, B. Upcroft, Simple Online and Realtime Tracking, in: IEEE International Conference on Image Processing (ICIP) 2016 (2016) 3464–3468, <https://doi.org/10.1109/ICIP.2016.7533003>.
- [32] N. Ma, X. Zhang, H.-T. Zheng, J. Sun, ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design, (2018). <https://doi.org/10.48550/arXiv.1807.11164>.
- [33] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, C. Xu, GhostNet: More Features from Cheap Operations (2020), <https://doi.org/10.48550/arXiv.1911.11907>.
- [34] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, in: Path Aggregation Network for Instance Segmentation, 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 8759–8768, <https://doi.org/10.1109/CVPR.2018.00913>.
- [35] P. Li, J. Zheng, P. Li, H. Long, M. Li, L. Gao, Tomato Maturity Detection and Counting Model Based on MHSA-YOLOv8, *Sensors* 23 (2023) 6701, <https://doi.org/10.3390/s23156701>.
- [36] T. Zheng, M. Jiang, Y. Li, M. Feng, Research on tomato detection in natural environment based on RC-YOLOv4, *Comput. Electron. Agric.* 198 (2022) 107029, <https://doi.org/10.1016/j.compag.2022.107029>.
- [37] N. El-Bendary, E. El Hariri, A.E. Hassanien, A. Badr, Using machine learning techniques for evaluating tomato ripeness, *Expert Syst. Appl.* 42 (2015) 1892–1905, <https://doi.org/10.1016/j.eswa.2014.09.057>.